

Reactコンポーネントの基本と組み立て方

ゴール：部品を組み合わせて画面を構成する考え方を理解する。

画面設計とコンポーネントの切り出し

自己紹介掲示板

名前

コメント

名前

コメント

名前

コメント

実装1：Propsによるデータの受け取り

UserCard.jsx

```
/*例: {name: "田中 太郎", comment: "数学が大好きです"}*/  
  
const UserCard = ({ name, comment }) => {  
  return (  
    <div>  
      <h3>{name}</h3>  
      <p>{comment}</p>  
    </div>  
  );  
};  
  
export default UserCard;
```

親画面からの呼び出し

page.jsx

```
import UserCard from "../component/UserCard";

const Home = () => {
  return <UserCard name="田中 太郎" comment="数学が大好きです" />;
};

export default Home;
```

実装2：CSS Modulesの導入

UserCard.module.css

```
.card {  
  border: 4px solid #e5e7eb;  
  padding: 16px;  
  border-radius: 12px;  
  margin: 32px;  
}  
.title {  
  font-size: 24px;  
}
```

CSS Modulesの適用

UserCard.jsx

```
import styles from "./UserCard.module.css";

const UserCard = ({ name, comment }) => {
  return (
    <div className={styles.card}>
      <h1 className={styles.title}>{name}</h1>
      <p>{comment}</p>
    </div>
  );
};

export default UserCard;
```

実装3：TSXへの変更と型定義

UserCard.tsx

```
interface UserCardProps {  
  name: string;  
  comment: string;  
}  
  
const UserCard = ({ name, comment }: UserCardProps) => {  
  return (  
    <div>  
      <h3>{name}</h3>  
      <p>{comment}</p>  
    </div>  
  );  
};  
  
export default UserCard;
```

実装4：複数のカードの呼び出し

page.tsx

```
import UserCard from "../component/UserCard";

const Home = () => {
  return (
    <div>
      <UserCard comment="数学が大好きです" name="田中 太郎"/>
      <UserCard comment="ラーメンが大好きです" name="田中 二郎"/>
      <UserCard comment="三河屋で働いています" name="田中 三郎"/>
    </div>
  );
};

export default Home;
```

実装5：リストコンポーネントの作成

CardList.tsx

```
import UserCard from "../UserCard";

interface CardListProps {
  cards: { id: string; name: string; comment: string }[];
}

const CardList = ({ cards }: CardListProps) => {
  return (
    <div>
      {cards.map((card) => {
        return (
          <UserCard comment={card.comment} key={card.id} name={card.name}/>
        );
      })}
    </div>
  );
};

export default CardList;
```

実装6：画面の完成

page.tsx

```
import CardList from "../component/CardList";

const users = [
  { id: "1", name: "田中 太郎", comment: "数学が大好きです" },
  { id: "2", name: "田中 二郎", comment: "ラーメンが大好きです" },
  { id: "3", name: "田中 三郎", comment: "三河屋で働いています" },
];

const Home = () => {
  return (
    <div>
      <CardList cards={users}/>
    </div>
  );
};

export default Home;
```

まとめと次回予告

- コンポーネントを組み合わせることで、より大きなコンポーネントを作れる。
- コンポーネントを切り出すことで、コードが整理され再利用が可能になる。
- 次の動画は全画面共通のヘッダー作成と、カードのクリックによるページ移動（ルーティング）を実装します。

補足資料1： export default

説明

理解しておいた方がよいことはdefaultという記述がある時とない時の差です。

defaultがついていると、そのコンポーネントファイルをimportしようとした時、defaultとしてexportされていたものがimportされます。

一方defaultがついていない場合、明示的に名前を指定してあげる必要があります。

ただし、defaultがついていないexportされたものはいくつでもimportすることができます。

書き方

呼び出されるコンポーネント側（例：CardList.tsx）にdefaultがある時、

```
import CardList from "../component/CardList";
```

というように呼び出し側(例:page.tsx)でimportできます。

しかし、`export const CardList` のようにdefaultがない場合、

```
import { CardList } from "../component/CardList";
```

というように{}をつける必要があります。複数をimportするときは、`{CardList, CardProperty}` というようにカンマで区切ります。

参考資料：

- React公式 - コンポーネントのインポートとエクスポート

<https://ja.react.dev/learn/importing-and-exporting-components>

補足資料2： JSX と TSX

JSX(.jsx)は、JavaScriptのファイル内にHTMLのようなタグの構文を直接書くことができる、React特有の機能（構文拡張）です。JavaScript XMLの略です。Reactを使わないWeb制作において、HTMLとJavaScriptのファイルは

```
index.html（画面の構造）  
script.js（画面の動き）
```

のように分かれています。JSXにおいてはHTMLのような構文をJavaScriptの中に直接かけるようになっています。

TSX(.tsx)は、そのJSXに対してTypeScriptの機能を追加したものです。

参考資料：

- React公式 - JSX でマークアップを記述する

<https://ja.react.dev/learn/writing-markup-with-jsx>